

Relational Algebra

It consists of a set of operations that take one or two relations as input and produce a new relation as their result.

It is a procedural query language

Input : One or more relations.
Output : A relation.

Fundamental Operations:

Select (σ)

Project (Π)

Rename (ρ)

Union ($\cup$)

Set difference ($-$)

Cartesian product ($\times$)

The select, project and rename operations are called unary operations, because they operate on one relation.

The other three operations operate on pairs of relations and therefore called binary operations.

ADDITIONAL OPERATIONS

- Intersection ($\cap$)
- Natural join ($\bowtie$)
- Assignment ($\leftarrow$)
- Inner Join
- Outer join

EXTENDED RELATIONAL ALGEBRA OPERATIONS

- Generalized projection
- Aggregation
- Division ($\div$)

SELECT:

Person

<i>Id</i>	<i>Name</i>	<i>Address</i>	<i>Hobby</i>
1123	John	123 Main	stamps
1123	John	123 Main	coins
5556	Mary	7 Lake Dr	hiking
9876	Bart	5 Pine St	stamps

$\sigma_{Hobby='stamps'}(Person)$

<i>Id</i>	<i>Name</i>	<i>Address</i>	<i>Hobby</i>
1123	John	123 Main	stamps
9876	Bart	5 Pine St	stamps

1)Get the details of Person whose id is greater than 3000 or whose hobby is hiking.

ANSWER:

$$\sigma_{Id > 3000 \text{ Or } Hobby = 'hiking'}(\text{Person})$$

2)

INSTRUCTOR			
ID	Name	Dept_Name	Salary
10101	John	Biology	65000
12121	Robin	Computer Science	90000
25252	Alya	Electrical	40000
26589	Yusuf	Finance	95000
54789	Ravi	Music	60000
78787	Raj	Physics	87000
87458	Jayant	History	75000
76985	Pratik	Computer Science	89000
12547	Neil	Finance	80000

Example 3: Find all instructors who are working in "Finance" department and drawing the salary greater than \$87,000. 

$$\sigma_{Dept_Name = 'Finance' \wedge Salary > 87000}(\text{INSTRUCTOR})$$

PROJECTION (Π)

-
Person

<i>Id</i>	<i>Name</i>	<i>Address</i>	<i>Hobby</i>
1123	John	123 Main	stamps
1123	John	123 Main	coins
5556	Mary	7 Lake Dr	hiking
9876	Bart	5 Pine St	stamps

$\Pi_{Name, Hobby}(\text{Person})$

<i>Name</i>	<i>Hobby</i>
John	stamps
John	coins
Mary	hiking
Bart	stamps

Note:

Duplicate rows are eliminated from the result.

Example:

$\Pi_{Name,Address}(\text{Person})$

<i>Name</i>	<i>Address</i>
John	123 Main
Mary	7 Lake Dr
Bart	5 Pine St

COMPOSITION OF OPERATIONS

$\Pi_{Id, Name} (\sigma_{Hobby='stamps' \text{ OR } Hobby='coins'} (\text{Person}))$

<i>Id</i>	<i>Name</i>	<i>Address</i>	<i>Hobby</i>
1123	John	123 Main	stamps
1123	John	123 Main	coins
5556	Mary	7 Lake Dr	hiking
9876	Bart	5 Pine St	stamps

Person

<i>Id</i>	<i>Name</i>
1123	John
9876	Bart

Result

Rename (ρ):

Syntax

$\rho_{\text{new_relation_name}}(\text{new_attribute_name1}, \text{new_attribute_name2}, \dots)(\text{relation})$

Where:

- **new_relation_name** is the new name you want to give to the relation (optional).
- **new_attribute_name1, new_attribute_name2, ...** are the new names for the columns (attributes).
- **relation** is the original relation (table).

Query to rename the table name Project to Pro and its attributes to P, Q, R.

```
 $\rho_{\text{Pro}}(\text{P}, \text{Q}, \text{R}) (\text{Project})$ 
```

UNION ($A \cup B$)

R and S must be the same number of attributes.

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

A	B
α	1
α	2
β	1
β	3

SET DIFFERENCE (A – B)

Includes all tuples that are in A but not in B

<i>A</i>	<i>B</i>
α	1
α	2
β	1

r

<i>A</i>	<i>B</i>
α	2
β	3

s

<i>A</i>	<i>B</i>
α	1
β	1

r – *s*

CARTESIAN PRODUCT (A X B)

Relations r, s :

A	B
α	1
β	2

r

C	D	E
α	10	a
β	10	a
β	20	b
γ	10	b

s

$r \times s$:

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

ADDITIONAL OPERATIONS - INTERSECTION ($A \cap B$)

Subject			Subject2			Result	
Name	Course	∩	Name	Course	➡	Name	Course
Systems	BCS		DataMining	MCS		Systems	BCS
Database	BCS		Database	MCS		Database	MCS
Database	MCS		Systems	BCS			
Algebra	MCS		Writing	BCS			

=> It should have same no of columns but can vary in rows

=> Then column heading should be also same

ASSIGNMENT ($\leftarrow$)

$$\begin{aligned} r_1 &\leftarrow \pi_{\text{name}}(\text{depositor}) \\ r_2 &\leftarrow \pi_{\text{name}}(\text{borrower}) \\ \text{result} &= r_1 - r_2 \end{aligned}$$

Relational Algebra - Join ($\bowtie$)

=>Join operation denoted by $\bowtie$.

- Inner Join
- Outer join:
 - Left Outer Join
 - Right Outer Join
 - Full Outer Join

Inner Join

Tables Before Joining

Students Table

Student_ID	Name	Age	Dept_ID
1	Alice	20	101
2	Bob	22	102
3	Charlie	21	103
4	David	23	105

Departments Table

Dept_ID	Dept_Name
101	CS
102	IT
103	ECE ↓

```
SELECT Students.Student_ID, Students.Name, Students.Age, Departments.Dept_Name
FROM Students
INNER JOIN Departments
ON Students.Dept_ID = Departments.Dept_ID;
```

Final Output After INNER JOIN

Student_ID	Name	Age	Dept_Name
1	Alice	20	CS
2	Bob	22	IT
3	Charlie	21	ECE

Natural Join

- Duplicate column(s) from the second table are removed.

Students Table

Student_ID	Name	Age	Dept_ID
1	Alice	20	101
2	Bob	22	102
3	Charlie	21	103
4	David	23	105

Departments Table

Dept_ID	Dept_Name
101	CS
102	IT
103	ECE

```
SELECT *  
FROM Students  
NATURAL JOIN Departments;
```

Final Result After Natural Join

Student_ID	Name	Age	Dept_ID	Dept_Name
1	Alice	20	101	CS
2	Bob	22	102	IT
3	Charlie	21	103	ECE

◆ Key Differences Between INNER JOIN and NATURAL JOIN

Feature	Inner Join	Natural Join
Condition Needed?	✓ Requires <code>ON table1.column = table2.column</code>	✗ No condition needed
Uses Common Column Names?	✗ No, needs explicit condition	✓ Yes, automatically matches by column names
Duplicate Columns?	✓ Keeps both columns	✗ Removes duplicate column from second table
Matching Required?	✓ Yes	✓ Yes

Outer Join

LEFT OUTER JOIN:

Employees Table

Emp_ID	Name	Dept_ID
1	Alice	101
2	Bob	102
3	Charlie	103
4	David	NULL

Departments Table

Dept_ID	Dept_Name
101	IT
102	HR
104	Finance

✓ Returns all rows from the left table (Employees), even if there's no match in the right table (Departments).

✓ If there is no match, NULL is returned for the right table.

```
SELECT Employees.Emp_ID, Employees.Name, Employees.Dept_ID, Departments.Dept_Name
FROM Employees
LEFT JOIN Departments
ON Employees.Dept_ID = Departments.Dept_ID;
```

Emp_ID	Name	Dept_ID	Dept_Name
1	Alice	101	IT
2	Bob	102	HR
3	Charlie	103	NULL
4	David	NULL	NULL

RIGHT OUTER JOIN (RIGHT JOIN)

✓ Returns all rows from the right table (Departments), even if there's no match in the left table (Employees).

✓ If there is no match, NULL is returned for the left table.

```
SELECT Employees.Emp_ID, Employees.Name, Employees.Dept_ID, Departments.Dept_Name
FROM Employees
RIGHT JOIN Departments
ON Employees.Dept_ID = Departments.Dept_ID;
```

Emp_ID	Name	Dept_ID	Dept_Name
1	Alice	101	IT
2	Bob	102	HR
NULL	NULL	104	Finance

FULL OUTER JOIN

- ✓ Returns all rows from both tables (Employees and Departments).
- ✓ If there's no match, NULL appears in missing columns.

```
SELECT Employees.Emp_ID, Employees.Name, Employees.Dept_ID, Departments.Dept_Name
FROM Employees
FULL OUTER JOIN Departments
ON Employees.Dept_ID = Departments.Dept_ID;
```

Emp_ID	Name	Dept_ID	Dept_Name
1	Alice	101	IT
2	Bob	102	HR
3	Charlie	103	NULL
4	David	NULL	NULL
NULL	NULL	104	Finance

Generalized Projection (π)

Emp_ID	Name	Salary	Bonus
1	Alice	50000	5000
2	Bob	60000	7000
3	Charlie	55000	6000

$\pi_{Emp_ID, Name, (Salary + Bonus) \rightarrow Total_Compensation}(Employees)$

Result:

Emp_ID	Name	Total_Compensation
1	Alice	55000
2	Bob	67000
3	Charlie	61000

Aggregation (γ)

Aggregation groups tuples based on attributes and applies aggregate functions such as:

- **SUM()** → Total sum of values
- **AVG()** → Average value
- **COUNT()** → Number of rows
- **MAX()** → Maximum value
- **MIN()** → Minimum value

Division:

sno	pno
s1	p1
s1	p2
s1	p3
s1	p4
s2	p1
s2	p2
s3	p2
s4	p2
s4	p4

A

pno
p2

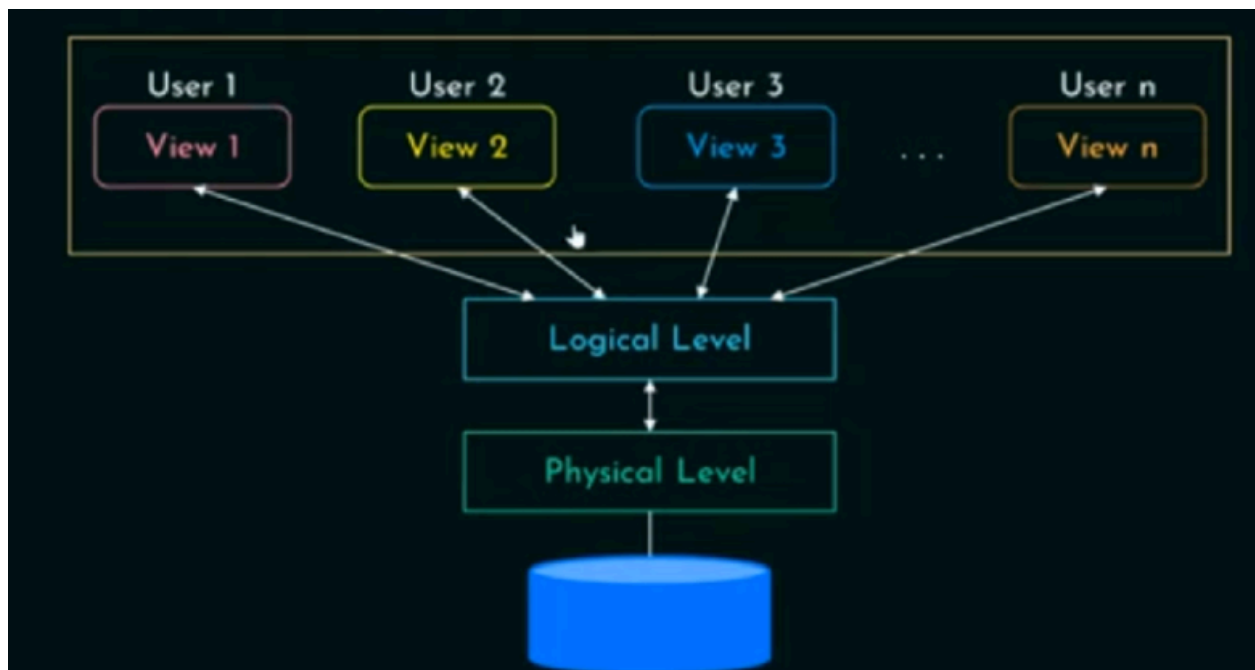
B1

sno
s1
s2
s3
s4

A÷B1

<https://chatgpt.com/share/67e58ae6-1020-8010-9d54-c6498f8fee41>

Views of Data:



What is Data Abstraction? (5 Simple Points)

- Data abstraction means hiding unnecessary details and showing only the needed information.

- **It hides complex information**
- **There are three levels of data abstraction: Physical, Logical, and View.**
- **Users interact with only the required data, making databases easier to use**
- **A database stores a lot of data, but users don't need to see all of it.**
- **Data abstraction hides unnecessary details and shows only the needed information.**
- **Example: A library has thousands of books, but you only search for the one you need.**
- **Users don't need to know how data is stored or managed, they just access what they need.**
- **This makes databases easier to use and understand.**

Physical Level (Lowest Level) – How Data is Stored

- **Describes how and where data is saved in memory.**
- **Includes files, indexing, and data storage techniques.**
- **Deals with internal database operations like compression and encryption.**
- **Users do not see this level; only database administrators manage it.**

- **Example: Data is stored in hard disks, SSDs, or cloud servers.**

Logical Level (Middle Level) – What Data is Stored

- **Describes what data exists in the database and how it is related.**
- **Uses tables, columns, and relationships to organize data.**
- **Does not show how data is stored physically.**
- **Provides logical data independence, Logical Data Independence means that if the way data is stored (e.g., on a hard disk, SSD, or cloud) changes, the structure of the database (tables, relationships, and queries) will remain the same.**
- **Example: A Student Table with Name, ID, and Course fields.**

3. View Level (Highest Level) – User Interaction

- **Shows only the necessary data to specific users.**
- **Hides complexity and protects sensitive information.**
- **Users interact with this level using applications or queries.**
- **Provides different views for different users (e.g., a teacher sees grades, a student sees personal details).**

- **Example: A Bank App where customers see only their account details, not the bank's full database.**